



UNIVERSITÉ JOSEPH KI-ZERBO
(UJKZ)

INSTITUT BURKINABÈ DES ARTS ET MÉTIERS
(IBAM)



Cours : Web Sémantique et Web de Données

CAHIER DES CHARGES TECHNIQUE

Moteur de recherche sémantique pour le mooré

Recherche Web multilingue : mooré, français, anglais

Présenté par

SAVADOGO Adjaratou

KOMPAORE Malick

OUEDRAOGO Abdoul Razack

KABORE Olivier

Encadré par

Dr. GUINKO T. Ferdinand

Année universitaire 2025–2026

Table des matières

Introduction générale	3
Le problème n'était pas celui que nous pensions	3
Notre démarche de travail	3
1. Architecture générale de la plateforme	5
1.1 Rappel du besoin et principe de conception	5
1.2 Architecture en couches	5
1.3 Trajet d'une requête utilisateur	5
2. Conception de la base vectorielle	7
2.1 Glossaire des notions employées	7
2.2 Le corpus source et sa structure	7
2.3 Démarche de vectorisation	7
2.4 Les cinq index et leur rôle	8
2.5 Fusion par les rangs	8
2.6 Routage automatique selon la langue	9
2.7 Priorité de la correspondance exacte	9
3. Traitement automatique de la langue	10
3.1 Normalisation du français	10
3.2 Index inverse et segmentation	10
3.3 Lemmatisation et mots-outils	10
3.4 Tournures idiomatiques	10
3.5 Découpage des domaines	11
4. Recherche sur le Web	12
4.1 Sources retenues et critères de choix	12
4.2 Le pont de traduction	12
4.3 Reclassement et exposition du mooré	12
5. Qualité des données du corpus	14
6. Spécifications techniques de mise en œuvre	15
6.1 Exposition par une API applicative	15
6.2 Sécurité et droits d'accès	15
6.3 Qualité, performance et compatibilité	15
6.4 Tests et validation	16
6.5 Gestion de versions et dépendances	16
7. Choix techniques et alternatives écartées	17
Conclusion	18
Annexe A : Résultats du banc d'essai	19
Annexe B : Contrats des points d'entrée	20
Références bibliographiques	22

Introduction générale

Le cahier des charges technique présente les choix retenus pour la réalisation du projet. Il précise les technologies employées, l'organisation du système, la structure des données et les éléments nécessaires à sa mise en place.

Dans le cadre du cours de Web sémantique et web de données, ce document porte sur le thème « Moteur de recherche sémantique pour le mooré ». Il s'agit de permettre à un utilisateur de formuler une requête en mooré, en français ou en anglais, et d'obtenir des pages Web accompagnées de leur titre, d'un extrait et d'un lien.

Le problème n'était pas celui que nous pensions

Nous avons abordé le sujet en croyant que la difficulté serait de chercher sur le Web. Elle est ailleurs. Les moteurs de recherche existent, ils fonctionnent bien, et rien ne justifierait d'en réécrire un pour un projet d'étudiants.

La vraie difficulté vient de la langue. Le mooré est une langue à faibles ressources : peu de contenus numériques, aucun outil de traitement automatique librement disponible, et dix caractères qui ne figurent pas sur un clavier ordinaire. Un moteur généraliste interrogé avec le mot `koom` ne rend rien d'utile, non parce qu'il serait mauvais, mais parce que ce mot ne lui dit rien. La question s'est donc déplacée : elle n'est pas de savoir comment chercher sur le Web, mais comment faire franchir à une requête mooré la barrière qui la sépare d'un Web francophone et anglophone.

Notre réponse tient dans une inversion de rôle. Nous disposions du dictionnaire *Webonary Moore*, soit 10 566 entrées mooré avec leurs définitions française et anglaise, et nous avons d'abord conçu le projet comme un moteur de recherche *dans* ce dictionnaire. Le cahier des charges fonctionnel dit pourtant explicitement que le système doit dépasser « une application limitée à un dictionnaire ». Nous avons donc changé la place du corpus : de destination, il est devenu **pont**. Une requête mooré est traduite avant d'être envoyée aux sources francophones, et une requête française est rapprochée de son équivalent mooré pour interroger les sources rédigées en mooré.

Notre démarche de travail

Nous avons travaillé par itérations courtes, en gardant une règle : ne rien décider sur une impression. Chaque fois qu'un choix se présentait, nous avons cherché à le mesurer avant de trancher. Cette discipline nous a coûté du temps, et elle nous a évité plusieurs erreurs que nous aurions autrement livrées sans les voir.

Le cas le plus net concerne les plongements lexicaux. Remplacer notre représentation TF-IDF par un modèle pré-entraîné paraissait l'amélioration évidente, et c'est d'ailleurs ce que nous avons annoncé dans une première version du *README*. La mesure a montré que ce modèle obtient **0,0 %** de réussite sur une requête en mooré, langue qu'il ne connaît pas. Le remplacement aurait amélioré une moitié du moteur en détruisant l'autre. Nous avons conservé les deux représentations.

Nous avons ensuite fait la même erreur à plus petite échelle, en prenant la fusion des deux index comme réglage par défaut. Là encore, la mesure a tranché contre nous : la fusion gagne nettement en français et perd en mooré. C'est de ce second constat qu'est né l'aiguillage par langue décrit en section 2.6.

Un troisième épisode mérite d'être rapporté parce qu'il porte sur la mesure elle-même. Notre banc d'essai indiquait 100 % de réussite pour tous les index sur la recherche d'un mot mooré. Ce résultat était trop beau : nous avons écrit un raccourci qui fait passer les correspondances exactes avant les vecteurs, et le test interrogeait le moteur avec le mot exact. Il ne mesurait donc plus que notre propre raccourci. Nous l'avons remplacé par une requête

volontairement mal orthographiée, ce qui force le passage par les vecteurs. Le score est tombé à 16,6 %, et c'est ce chiffre honnête qui nous a conduits à construire un index dédié aux en-têtes.

La suite du document présente l'architecture générale, la conception de la base vectorielle, le traitement de la langue, la recherche Web, la qualité des données, puis les spécifications de mise en œuvre. Une section est consacrée aux solutions que nous avons envisagées puis écartées, avec la mesure qui a motivé chaque arbitrage. Tous les chiffres cités sont reproductibles par la commande `python3 benchmark.py`.

1. Architecture générale de la plateforme

1.1 Rappel du besoin et principe de conception

Le cahier des charges fonctionnel demande un moteur de recherche Web multilingue : saisie en mooré, en français ou en anglais, résultats provenant du Web, affichés sous forme de titres, d'extraits et de liens, avec une amélioration progressive de la pertinence par une approche sémantique.

Le principe de conception qui structure toute la solution tient en une phrase : **le corpus mooré n'est pas la destination de la recherche, il en est le pont**. Interroger un moteur francophone avec le mot mooré `koom` ne donne rien ; le traduire en « eau » avant de l'envoyer change tout. Réciproquement, une requête française est rapprochée de son équivalent mooré pour interroger les sources rédigées en mooré.

Conséquence de conception. Le dictionnaire vectoriel n'est pas un module annexe : il est placé sur le chemin critique de chaque requête Web. Sa qualité conditionne directement la pertinence des pages rendues.

1.2 Architecture en couches

Couche présentation : page HTML/CSS/JavaScript sans framework

Trois modes : recherche Web, consultation du dictionnaire, glossage de phrases. Clavier virtuel pour les dix caractères propres au mooré.

▲ JSON ▼

Couche service : API Flask (`api.py`)

Validation des paramètres, restriction des origines, sérialisation JSON, service de la page d'interface.

▲ ▼

Couche métier : quatre modules Python

`search.py` (cinq index et leur routage) · `translate.py` (segmentation et glossage) · `recherche_web.py` (pont et sources externes) · `liens.py` (ressources de consultation).

▲ ▼

Couche données : corpus et index

Corpus CSV (10 566 entrées) · index FAISS TF-IDF/LSA · index de caractères sur les en-têtes · index d'embeddings (optionnel).

▲ HTTPS ▼

Sources externes : API MediaWiki

Wikipédia en mooré · Wikipédia en français · Wiktionnaire. Interrogées en parallèle, sans clé d'API.

1.3 Trajet d'une requête utilisateur

L'exemple ci-dessous suit la requête `koom` (« eau » en mooré) depuis la saisie jusqu'à l'affichage.

1. Saisie	l'utilisateur tape « koom » dans l'onglet Web
2. Requête HTTP	GET /api/web?q=koom&k=10
3. Détection	search.detecter_langue("koom") -> « moore » (le mot figure dans le vocabulaire des en-têtes)
4. Pont	search.search("koom", backend="entrees") -> koom, score 1,000 définition française retenue : « eau » seuil de confiance 0,45 respecté -> traduction acceptée
5. Interrogation	trois appels HTTPS parallèles : mos.wikipedia.org srsearch=koom fr.wikipedia.org srsearch=eau fr.wiktionary.org srsearch=eau
6. Normalisation	suppression du balisage , décodage des entités HTML, reconstruction des URL
7. Reclassement	encodage de « eau » et de chaque couple titre+extrait, tri par similarité cosinus, SAUF les pages en mooré
8. Entrelacement	une page en mooré toutes les trois positions
9. Réponse JSON	{ pont, sources, resultats: [{titre, extrait, url, ...}] }
10. Affichage	titre cliquable, extrait, source, terme interrogé

Les étapes 3 et 4 constituent le pont : elles n'existent dans aucun moteur généraliste et sont la contribution propre du projet.

2. Conception de la base vectorielle

2.1 Glossaire des notions employées

Tableau 2.1 : Notions techniques employées dans ce document.

Notion	Définition retenue
Vecteur	Représentation numérique d'un texte, permettant de mesurer une proximité entre textes
TF-IDF	Pondération donnant du poids aux termes fréquents dans un document et rares dans le corpus
n-gramme de caractères	Fragment de n caractères consécutifs ; capte la morphologie et tolère les fautes
LSA / SVD	Réduction du nombre de dimensions d'une matrice creuse, ici de 56 529 à 256
Embedding	Vecteur produit par un modèle pré-entraîné, porteur du sens et non de la forme
Similarité cosinus	Mesure de proximité entre deux vecteurs, comprise entre -1 et 1
Index FAISS	Structure de recherche des vecteurs les plus proches d'un vecteur donné
RRF	Fusion de plusieurs classements en n'utilisant que le rang, jamais le score
Glose	Équivalent mot à mot, sans reconstruction de la grammaire de la langue cible

2.2 Le corpus source et sa structure

Le corpus provient du dictionnaire *Webonary Moore* (SIL). Il est livré au format CSV, séparateur point-virgule, encodage UTF-8, et compte **10 566 entrées** réparties en onze colonnes.

Tableau 2.2 : Colonnes du corpus et taux de remplissage mesurés.

Colonne	Remplissage	Usage dans le moteur
mot_moore	100 %	En-tête indexé ; sert de terme mooré au pont
definition_francais	100 %	Index inverse français ; terme envoyé au Web
definition_english	99,3 %	Index sémantique ; requêtes en anglais
categorie_grammaticale	100 %	Filtrage des mots-outils, catégories idiomatiques
prononciation	57,4 %	Affichage
domaine	35,4 %	Filtrage par champ sémantique (52 domaines)
pluriel	30,4 %	Affichage des renvois
synonymes	24,6 %	Renvois cliquables
variante	14,6 %	Renvois cliquables
sens_num	21,4 %	Distinction des sens multiples (947 mots concernés)
antonymes	1,5 %	Renvois cliquables

Le cahier des charges fonctionnel anticipait que les champs secondaires seraient peu renseignés et proposait de traiter leur exploitation comme une évolution. La mesure confirme le diagnostic (1,5 % pour les antonymes), mais ces champs ont malgré tout été exploités, l'interface se contentant de les omettre lorsqu'ils sont vides.

2.3 Démarche de vectorisation

Le programme `build_index.py` construit la représentation vectorielle du corpus en quatre étapes.

1. **Composition du document.** Pour chaque entrée, un texte unique est formé en concaténant le mot mooré, ses définitions française et anglaise, sa variante et son domaine.
2. **Double vectorisation TF-IDF.** Un vectoriseur en n-grammes de caractères (2 à 4, mode `char_wb`) capte la morphologie du mooré et ses diacritiques ; un vectoriseur en mots (1 à 2) capte le sens porté par les définitions. Les deux matrices sont concaténées avec une pondération de 0,5 pour les caractères et 1,0 pour les mots.
3. **Réduction LSA.** La matrice creuse obtenue ($10\,566 \times 56\,529$) est réduite à 256 dimensions par décomposition en valeurs singulières.
4. **Normalisation et indexation.** Les vecteurs sont normalisés, ce qui rend le produit scalaire égal à la similarité cosinus, puis chargés dans un index FAISS de type `IndexFlatIP`.

La même transformation est appliquée à chaque requête, garantissant que requête et documents vivent dans le même espace.

2.4 Les cinq index et leur rôle

L'expérience a montré qu'aucun index unique ne convient aux deux langues du projet. Cinq sont donc disponibles, dont un mode automatique.

Tableau 2.3 : Les cinq index de recherche.

Index	Ce qu'il indexe	Point fort / limite
entrees	Les en-têtes mooré seuls, n-grammes de caractères, construit à la volée	Retrouve un mot mooré même mal orthographié. Ignore les définitions.
tfidf	Document mixte : en-tête et définitions	Polyvalent, sans dépendance. Ne fait pas de sémantique.
semantique	Définitions fr/en, embeddings multilingues pré-entraînés	Comprend le sens en français et en anglais. Ignore le mooré.
hybride	Fusion RRF de tfidf et semantique	Meilleur résultat sur requête française. Dégrade le mooré.
auto	Aiguillage vers l'un des précédents selon la langue détectée	Mode par défaut. Seul à bien traiter les deux langues.

Le modèle d'embeddings ne connaît pas le mooré. Le modèle retenu, paraphrase-multilingual-MiniLM-L12-v2, couvre une cinquantaine de langues parmi lesquelles le mooré, langue gur à faibles ressources, ne figure pas. Son tokenizer découpe les mots mooré en sous-mots dépourvus de signification pour lui. Mesuré, l'index sémantique obtient **0,0 %** de réussite sur une requête mooré. C'est la raison pour laquelle le remplacement pur du TF-IDF par des embeddings, solution qui paraissait naturelle, a été écarté.

2.5 Fusion par les rangs

Fusionner deux classements suppose de comparer leurs scores. Or un cosinus TF-IDF/LSA et un cosinus d'embeddings ne vivent pas sur la même échelle, et les normaliser demanderait une calibration arbitraire. La méthode retenue, la *Reciprocal Rank Fusion*, n'utilise que le rang :

$$\text{score}(\text{document}) = \sum_i \frac{1}{k + \text{rang dans la liste } i} \quad \text{avec } k = 60$$

Un document bien classé par les deux moteurs devance un document excellent chez l'un et absent chez l'autre. La méthode ne comporte aucun paramètre à ajuster sur les données, ce qui écarte tout risque de surapprentissage sur un corpus de cette taille.

2.6 Routage automatique selon la langue

Le mode `auto` choisit l'index d'après la langue de la requête. La détection n'emploie aucune bibliothèque externe : sur des requêtes d'un ou deux mots, elles sont peu fiables, alors que le corpus fournit directement la réponse.

1. Si la requête contient l'un des caractères propres au mooré (ã ã ì ò ù é ı v ŋ ɲ), elle est mooré.
2. Sinon, les mots de la requête sont comptés dans deux vocabulaires extraits du corpus : celui des en-têtes et variantes d'une part, celui des définitions française et anglaise d'autre part.
3. Si aucun mot n'appartient à l'un ou l'autre, la requête est traitée comme du mooré : un terme inconnu des deux côtés est le plus souvent une entrée mooré mal orthographiée, cas où seul l'index lexical peut aider.

L'aiguillage envoie ensuite le mooré vers `entrees` et le français ou l'anglais vers `hybride`, ou vers `tfidf` lorsque l'index sémantique n'a pas été construit.

2.7 Priorité de la correspondance exacte

Ni le TF-IDF ni les embeddings ne garantissent qu'une entrée cherchée mot pour mot sorte en tête : les vecteurs sont compressés par la réduction LSA, et une entrée courte comme `koom` se faisait dépasser par `ruud-koom` ou `koom guls moodo`, dont le document est plus riche. Or ne pas rendre en premier le mot exactement demandé est le défaut le plus visible qu'un moteur de dictionnaire puisse présenter.

Une table des en-têtes normalisés est donc consultée avant les vecteurs, et ses résultats sont placés en tête quel que soit l'index actif. Elle tolère l'absence de diacritiques : `bag-teoko` retrouve `bāg-tēoko`, ce qui compte pour un utilisateur dont le clavier ne porte pas ces caractères.

3. Traitement automatique de la langue

3.1 Normalisation du français

Toute comparaison entre une requête française et le corpus passe par une normalisation. Deux traitements se sont révélés indispensables, chacun après constat d'une erreur.

- **Élision.** « l'eau » doit être découpé en « l' » et « eau », faute de quoi le mot reste introuvable. Les élisions `c' d' j' l' m' n' s' t' qu'` sont séparées.
- **Ligatures.** Les caractères « œ » et « æ » n'appartiennent pas à la plage latine de base ; traités comme de la ponctuation, ils étaient supprimés, et « œil » devenait « il ». Ils sont désormais convertis en « oe » et « ae ».

Deux formes normalisées sont conservées : l'une avec accents, l'autre sans. La première est essayée en premier, car les accents portent le sens : « marché » désigne le lieu (*raaga*) et « marche » l'action (*kēnde*). La seconde sert de rattrapage.

3.2 Index inverse et segmentation

Le glossage d'une phrase repose sur un index inverse construit à partir des définitions françaises. Une définition comme « manger, consommer » donne deux clés distinctes, afin que l'entrée soit retrouvée par l'un ou l'autre terme.

Tous les fragments ne sont pas des gloses. Découper les définitions sur les virgules produit les synonymes voulus, mais découpe aussi les définitions descriptives en fragments qui n'en sont pas : « dispositif pour sécher ou griller la viande, le poisson » produisait la clé « le poisson », et la phrase « il a mangé le poisson » y tombait. Règle retenue : le premier fragment est toujours une glose ; les suivants ne le sont que s'ils ne commencent pas par un mot de liaison. **220 clés parasites** ont ainsi été écartées.

La segmentation procède ensuite par correspondance la plus longue : sur une suite de mots, les expressions figées sont cherchées avant les mots isolés. Le corpus contient 504 expressions et 5 753 gloses françaises de plus d'un mot, ce qui rend cette priorité déterminante : « n'importe comment » doit rendre *a bal* d'un bloc et non « n'importe » suivi de « comment ».

3.3 Lemmatisation et mots-outils

Les formes conjuguées sont ramenées à leur infinitif par une table des irréguliers les plus fréquents complétée par un découpage des terminaisons régulières : « veux » devient « vouloir », « mangé » devient « manger ».

Les articles et prépositions français sont volontairement laissés non traduits. Le mooré n'a pas d'articles, et les rapports que le français rend par une préposition y sont exprimés par postposition ou par simple juxtaposition. Les chercher produit des faux amis : « la » rendait *ka*, qui est la particule de négation. Un garde-fou complémentaire filtre par catégorie grammaticale les mots-outils ambigus : sans lui, « pas » rendait *kēndre*, le nom « enjambée », et « est » rendait *yaanga*, le point cardinal.

3.4 Tournures idiomatiques

Une glose littérale ne peut pas deviner qu'une expression se dit autrement : « tu vas bien » se dit *laafi bala*, littéralement « ça va », et aucun mot ne coïncide. La base vectorielle est donc interrogée sur la phrase entière, et les entrées de type expression ou interjection qui s'en rapprochent sont proposées en complément de la glose.

Chaque index échouait ici d'une manière différente, ce qui a conduit à les combiner :

Tableau 3.1 : Comportement des index sur la recherche de tournures.

Index	Comportement observé
tfidf	Classe bien (<i>laafi bala</i> au rang 1) mais invente sur les phrases descriptives : « la maison de mon père » lui inspirait une tournure sans rapport, à 0,72.
semantique	Ne propose rien sur ces mêmes phrases (le bruit y tombe entre 0,03 et 0,35), mais classe mal : les définitions d'un ou deux mots obtiennent des scores élevés partout, et <i>laafi bala</i> ne sortait qu'au rang 3.
retenu	Ordre par fusion RRF , qui exige l'accord des deux index et remet <i>laafi bala</i> au rang 1, et filtre par le cosinus sémantique (seuil 0,55), qui élimine le bruit lexical. Le score affiché est ce cosinus, seul interprétable.

Ce seuil est calibré sur une poignée de phrases, faute de jeu de test annoté. Les suggestions sont présentées à l'utilisateur comme étant à valider par un locuteur ; elles sont fiables sur les formules courantes et bruitées sur les phrases descriptives.

3.5 Découpage des domaines

La colonne **domaine** mêle des valeurs simples (« Weather ») et des valeurs composées (« Agriculture, Water »). Un découpage sur la virgule serait faux, car certains domaines en contiennent une : « Bush, shrub », « Grass, herb, vine ».

Le vocabulaire des **52 domaines atomiques** est donc construit à partir du corpus, puis chaque valeur est découpée par correspondance la plus longue. Le procédé a été validé sur les 130 valeurs distinctes : toutes se reconstruisent à l'identique. Le filtrage est appliqué côté serveur avant la sélection des meilleurs résultats ; il fait passer la proportion d'entrées atteignables par un filtre de **63 % à 100 %**.

4. Recherche sur le Web

4.1 Sources retenues et critères de choix

Trois sources sont interrogées, toutes fondées sur le moteur MediaWiki. Le critère déterminant a été la disponibilité d'une **API publique sans clé ni quota**, condition pour que le projet reste exécutable par n'importe quel membre du groupe ou par l'enseignant.

Tableau 4.1 : Sources Web interrogées.

Source	Langue interrogée	Apport
Wikipédia en mooré	mooré	Contenus rédigés en mooré (1 326 articles)
Wikipédia en français	français	Couverture encyclopédique large
Wiktionnaire	français	Définitions, étymologie, formes fleuries

Les trois appels sont émis **en parallèle** ; les réponses sont mises en cache pour la durée du processus. Chaque appel est protégé : une source indisponible ne fait pas échouer la recherche, elle est simplement absente du résultat.

L'architecture est enfichable : ajouter un moteur généraliste (Bing, Brave, Google Custom Search) consiste à déclarer une entrée supplémentaire dans la liste des sources. Ces moteurs exigent une clé d'API, raison pour laquelle ils ne sont pas activés par défaut. **La couverture actuelle se limite donc à trois encyclopédies et n'est pas une exploration de tout le Web** ; cette limite est également énoncée dans le manuel d'utilisation.

4.2 Le pont de traduction

Le pont détermine, pour une requête donnée, le terme à envoyer aux sources mooré et celui à envoyer aux sources francophones.

- **Requête mooré.** Le mot part tel quel vers le Wikipédia mooré ; sa première glose française part vers les sources francophones.
- **Requête française ou anglaise.** Elle part telle quelle vers les sources francophones, et son équivalent mooré le plus proche part vers le Wikipédia mooré.

Le pont doit pouvoir refuser de traduire. La recherche vectorielle renvoie toujours un plus proche voisin, même mauvais : un mot inexistant comme `zzzqxwv` recevait la « traduction » « presque totalité », et le Web était interrogé sur cette invention. Un seuil de **0,45** sur le score de correspondance y met fin. Il est calibré sur des mesures : un mot exact obtient 1,000, une faute de frappe plausible 0,551, du charabia 0,256. En dessous du seuil, le terme est envoyé tel quel et l'interface l'indique.

4.3 Reclassement et exposition du mooré

Les résultats renvoyés par les sources sont classés par correspondance de mots. Un reclassement sémantique leur est appliqué : la requête et chaque couple titre-extrait sont encodés par le modèle d'embeddings, puis triés par similarité cosinus. C'est l'exigence d'amélioration de la pertinence par une approche sémantique, appliquée aux résultats Web.

Le reclassement ne doit pas s'appliquer au mooré. Le modèle ignorant cette langue, reclasser l'ensemble enterrerait systématiquement les pages du Wikipédia mooré : sur la requête koom , **aucune ne subsistait parmi les quinze premiers résultats**. Dans un moteur destiné à des locuteurs du mooré, c'est un contresens. Seules les pages en langue connue du modèle sont donc reclassées ; les pages en mooré conservent l'ordre de pertinence de leur propre source et sont intercalées à raison d'une position sur trois.

Ce quota est un compromis : suffisant pour que la langue du projet reste visible, assez mesuré pour ne pas évincer les résultats francophones lorsque le Wikipédia mooré, avec ses 1 326 articles, n'a rien de pertinent à offrir.

5. Qualité des données du corpus

Le corpus source comporte des défauts hérités de son extraction. Ils ont été recensés puis, lorsque c'était possible sans inventer de données, corrigés à l'affichage. Les corrections sont appliquées à la lecture et non dans le fichier CSV, afin que celui-ci reste identique à sa source.

Tableau 5.1 : Défauts relevés et traitements appliqués.

Défaut	Ampleur	Traitement
En-têtes mêlant le mot mooré et sa traduction : 'Come! y', 'why? bōeedga'	7 entrées	Extraction du segment mooré. Le mot suit la dernière ponctuation, sauf si la chaîne se termine par « ! » ou « ? », auquel cas il la précède.
Colonnes synonymes et antonymes contaminées l'une par l'autre : 'saamba; antonyme: ma1'	57 entrées	Démêlage par étiquette et redistribution dans la bonne colonne
Renvois suffixés d'un numéro de sens : ma1 pour ma	647 renvois	Suppression du chiffre final ; le taux de résolution passe de 2 800 à 3 429 sur 3 545
Définitions tronquées après une abréviation : « tētu (lit », « tomber (ex »	74 définitions	Le texte manquant est absent du CSV, donc irrécupérable. Un marqueur « [...] » signale la donnée incomplète.
Fragments de définition indexés comme gloses autonomes	220 clés	Règle du premier fragment (voir § 3.2)
Ligne aux colonnes décalées, avec résidu de scraping Category:	1 ligne	Trop ambiguë pour une réparation automatique. Écartée des suggestions ; correction manuelle à prévoir dans le CSV.
Doublons d'entrées	quelques cas	Non traités : deux entrées identiques restent deux entrées valides du dictionnaire

6. Spécifications techniques de mise en œuvre

6.1 Exposition par une API applicative

L'application Flask expose six points d'entrée en JSON et sert la page d'interface. Les contrats détaillés figurent en annexe B.

Tableau 6.1 : Points d'entrée de l'API.

Route	Rôle
GET /	Sert l'interface web
GET /api/web	Recherche de pages sur le Web (pont, sources, reclassement)
GET /api/search	Recherche dans le corpus (index, domaine, k)
GET /api/translate	Glossage d'une phrase et tournures idiomatiques
GET /api/liens	Ressources externes de consultation pour un mot
GET /api/domaines	Liste des 52 domaines déduits du corpus
GET /api/health	État du service, index disponibles, taille du corpus

6.2 Sécurité et droits d'accès

L'application est destinée à un usage local et ne comporte pas d'authentification, aucune donnée personnelle n'étant manipulée. Quatre mesures ont néanmoins été prises.

- **Origines restreintes.** Le partage entre origines est limité à `null` (page ouverte depuis le disque) et aux adresses locales. Sans cette restriction, tout site consulté pendant que l'API tourne pourrait l'interroger.
- **Débogueur désactivé.** Le mode debug de Flask expose une console d'exécution ; il est désactivé.
- **Validation des paramètres.** Le nombre de résultats demandé est contrôlé et borné : les valeurs `0`, `-1` et `abc` provoquaient une erreur 500 et rendent désormais une erreur 400 explicite. Les longueurs de requête sont plafonnées.
- **Échappement du contenu.** Toutes les données issues du corpus et des sources Web sont échappées avant insertion dans la page. Le corpus contient déjà une entrée porteuse d'une esperluette, et les extraits Web proviennent de sites tiers.

6.3 Qualité, performance et compatibilité

Les index et vectoriseurs, qui représentent environ 70 Mo, sont chargés une seule fois au démarrage du processus. Ils étaient auparavant relus à chaque requête, ce qui coûtait plus d'une demi-seconde par recherche.

Tableau 6.2 : Latences mesurées, modèle chargé, k = 20.

Index	Latence	Observation
entrees	20 – 25 ms	Le plus rapide
semantique	96 – 115 ms	Plus rapide que TF-IDF
tfidf	200 – 530 ms	La transformation SVD est le goulot d'étranglement
hybride	300 – 530 ms	Deux recherches puis fusion
auto	400 – 540 ms	Détection de langue puis aiguillage

recherche Web	dépend des sources	Trois appels parallèles, réponses mises en cache
---------------	--------------------	--

Compatibilité. Python 3.9 ou plus, testé sous Python 3.11 et Windows. L'application est multiplateforme et ne requiert ni serveur de base de données ni service externe. L'index sémantique est optionnel : en son absence, le mode automatique se limite aux index lexicaux et aucune fonctionnalité ne cesse d'opérer, hormis le reclassement sémantique.

6.4 Tests et validation

La validation repose sur trois dispositifs complémentaires.

1. **Tests de non-régression** : `test_moteur.py`, 66 tests, sans dépendance supplémentaire. Les tests exigeant l'index sémantique ou un accès réseau se sautent d'eux-mêmes. Ils couvrent les points où le corpus piège : domaines composés, en-têtes corrompus, colonnes contaminées, élisions et ligatures, fragments de définition, bornes des paramètres, restriction des origines.
2. **Validation par mutation** : cinq régressions ont été introduites volontairement dans le code pour vérifier que la suite les détecte : découpage naïf des domaines, arrêt du démêlage des renvois, article « la » de nouveau traduit, fragments réindexés, en-tête corrompu mal réparé. **Les cinq ont été détectées.** Une suite verte ne prouvant rien par elle-même, cette étape établit qu'elle mesure bien quelque chose.
3. **Banc d'essai chiffré** : `benchmark.py` compare les cinq index sur trois tâches, le corpus servant de vérité terrain. Les résultats figurent en annexe A.

6.5 Gestion de versions et dépendances

Le code est versionné sous Git, sur la branche `feature/base-vectorielle` du dépôt du groupe. Les contributions passent par cette branche puis par une *pull request* vers `main`, afin que le groupe puisse relire.

Tableau 6.3 : Dépendances.

Bibliothèque	Statut	Rôle
pandas, numpy, scipy	requis	Manipulation du corpus et des matrices
scikit-learn	requis	TF-IDF et réduction LSA
faiss-cpu	requis	Recherche des plus proches voisins
flask, flask-cors	requis	API et service de l'interface
sentence-transformers	optionnelle	Index sémantique et reclassement

L'index sémantique construit (15,5 Mo) n'est pas versionné : il est inutilisable sans la bibliothèque qui permet d'encoder la requête, et se reconstruit par une commande. Les index lexicaux, eux, sont fournis construits dans le dépôt afin que l'application soit utilisable immédiatement.

7. Choix techniques et alternatives écartées

Cette section consigne les décisions qui auraient pu être prises autrement. Nous la faisons figurer parce qu'un choix technique ne se juge pas seulement sur ce qu'il apporte, mais sur ce qu'il écarte et pour quelle raison. Chaque arbitrage rapporté ici s'appuie sur une mesure, et plusieurs d'entre eux vont contre notre intuition de départ.

Nous avons remarqué en écrivant cette section que les quatre premières lignes du tableau ont un point commun : dans les quatre cas, la solution écartée était celle qui paraissait la plus moderne ou la plus élégante. Le modèle pré-entraîné, la fusion systématique, la normalisation des scores et le recours à un moteur commercial sont tous des choix défendables ailleurs. Ils ne conviennent pas ici parce que notre langue de travail est peu dotée, et c'est cette contrainte, plus que la technique elle-même, qui a décidé.

Tableau 7.1 : Alternatives envisagées puis écartées.

Alternative	Raison de l'écartement
Remplacer le TF-IDF par des embeddings pré-entraînés	Le modèle ignore le mooré : 0,0 % de réussite sur une requête mooré contre 6,4 % pour le TF-IDF et 59,1 % pour l'index des en-têtes. Le remplacement aurait amélioré une moitié du moteur en détruisant l'autre.
Utiliser l'index hybride comme mode par défaut	Il gagne nettement en français (77,5 % contre 52,3 %) mais dégrade le mooré (13,2 % contre 16,6 % en top-5). Un défaut unique ne convenait pas ; d'où le routage par langue.
Fusionner les classements par combinaison des scores	Les échelles ne sont pas comparables et leur normalisation aurait exigé une calibration arbitraire sur un corpus réduit. La fusion par les rangs n'a aucun paramètre à ajuster.
Employer un moteur de recherche généraliste	Bing, Brave et Google Custom Search exigent une clé d'API et imposent des quotas. Le projet devait rester exécutable sans inscription. L'architecture prévoit leur ajout ultérieur.
Utiliser une bibliothèque de détection de langue	Peu fiables sur des requêtes d'un ou deux mots, et aucune ne connaît le mooré. Le vocabulaire du corpus fournit directement la réponse.
Reconstruire une phrase mooré grammaticale	Le mooré a son propre ordre des mots, ses postpositions et ses marques d'aspect. Un dictionnaire ne permet pas de les reconstruire. Le résultat est présenté comme une glose, et l'interface le précise.
Corriger les défauts directement dans le CSV	Le corpus doit rester identique à sa source pour rester vérifiable. Les corrections sont appliquées à la lecture, et documentées.

Conclusion

Ce cahier des charges technique a précisé les choix retenus pour la conception du moteur de recherche mooré. Il a présenté l'architecture en couches, la conception de la base vectorielle, le traitement automatique appliqué au français et au mooré, la recherche sur le Web, ainsi que les éléments de mise en œuvre : interface de programmation, sécurité, mesures de performance et validation.

Le principe qui organise l'ensemble est l'inversion du rôle du corpus. Employé comme destination, le dictionnaire aurait produit une application de consultation, ce que le cahier des charges fonctionnel écarte explicitement. Employé comme pont, il permet à une requête mooré d'atteindre des contenus qui ne sont pas rédigés dans cette langue. C'est cette bascule qui rend le moteur multilingue plutôt que littéral, et c'est elle que nous présenterions en premier si une seule idée devait être retenue de ce travail.

La mesure a occupé une place que nous n'avions pas prévue. Trois décisions importantes ont été prises contre notre intuition initiale, et dans les trois cas c'est un chiffre qui a tranché : le modèle pré-entraîné qui échoue totalement sur le mooré, la fusion des index qui dégrade ce qu'elle devait améliorer, et un banc d'essai qui affichait un résultat parfait parce qu'il ne mesurait plus rien. Nous retenons de ce projet que la difficulté d'une langue peu dotée ne se traite pas en appliquant les méthodes habituelles avec plus de soin, mais en vérifiant à chaque étape qu'elles s'appliquent encore.

Trois limites subsistent et sont assumées. Les sources Web se réduisent à trois encyclopédies, faute d'accès à un moteur généraliste sans clé d'interface. Le glossage d'une phrase produit un alignement mot à mot et non une phrase mooré grammaticale, ce qu'un dictionnaire ne permet pas de reconstruire. L'interface enfin n'existe qu'en français, alors que le public visé par le cahier des charges fonctionnel comprend des locuteurs du mooré ne maîtrisant pas nécessairement le français écrit ; c'est, de nos trois limites, celle qui nous paraît la plus contradictoire avec l'esprit du projet, et la première à traiter.

Les prolongements naturels sont l'ajout d'un moteur généraliste comme quatrième source, la reconnaissance vocale prévue par le cahier des charges fonctionnel en priorité basse, la traduction de l'interface en mooré, et l'extension à d'autres langues nationales du Burkina Faso. L'architecture a été conçue pour les accueillir : les sources Web et les index de recherche sont enfichables, et ajouter l'un ou l'autre revient à déclarer une entrée supplémentaire.

Annexe A : Résultats du banc d'essai

Mesures obtenues par `python3 benchmark.py`. La vérité terrain est le corpus lui-même ; ces chiffres évaluent donc la qualité de **récupération** et non la généralisation à des formulations inédites.

Tableau A.1 : Comparaison des cinq index sur deux tâches.

Index	Expression retrouvée depuis sa définition française (306 cas)		Entrée mooré retrouvée malgré une faute de frappe (235 cas)	
	top-1	top-5	top-1	top-5
entrees	0,7 %	2,9 %	59,1 %	77,0 %
tfidf	52,3 %	77,8 %	6,4 %	16,6 %
semantique	73,9 %	90,8 %	0,0 %	0,0 %
hybride	77,5 %	94,8 %	5,1 %	13,2 %
auto	77,5 %	94,8 %	57,4 %	74,9 %

Lecture : chaque index est excellent d'un côté du tableau et faible de l'autre. Le mode `auto` est le seul à obtenir de bons résultats dans les deux colonnes, ce qui justifie le routage par langue plutôt que le choix d'un index unique. L'écart entre `auto` et `entrees` sur le mooré (57,4 % contre 59,1 %) est le coût des requêtes mal aiguillées : un mot déformé ressemble parfois assez à du français pour partir du mauvais côté.

Gain apporté par la segmentation

Tableau A.2 : Retrouver une expression mooré depuis sa définition (306 cas).

Méthode	top-1	top-3
Vecteur unique sur la phrase entière	52,3 %	72,9 %
Segmentation, expressions d'abord	82,4 %	93,5 %

L'index de gloses étant construit sur ces mêmes définitions, cette mesure évalue la qualité de la segmentation et non la généralisation à des tournures inédites.

Annexe B : Contrats des points d'entrée

GET /api/web

```
Paramètres  q  requête (obligatoire, 200 caractères maxi)
             k  nombre de résultats (1 à 20, défaut 8)

Réponse     {
  "requete": "koom",
  "terme_moore": "koom",
  "terme_francais": "eau",
  "pont": "« koom » traduit en « eau » par le corpus",
  "sources": ["Wikipédia en mooré", "Wikipédia", "Wiktionnaire"],
  "reclasse_semantiquement": true,
  "resultats": [
    { "titre": "Eau",
      "extrait": "précise : eau minérale, eau de Seltz...",
      "url": "https://fr.wikipedia.org/wiki/Eau",
      "source": "Wikipédia",
      "source_id": "fr.wikipedia",
      "terme_interroge": "eau",
      "score": 0.65 }
  ]
}
```

Erreurs 400 requête trop longue, ou k invalide

GET /api/search

```
Paramètres  q      requête (obligatoire)
             k      nombre de résultats (1 à 100, défaut 10)
             domaine restriction à un domaine (facultatif)
             backend entrees | tfidf | semantique | hybride | auto

Réponse     [ { "mot_moore", "definition_francais", "definition_english",
                "categorie", "prononciation", "domaine",
                "synonymes": [], "antonymes": [], "variantes": [],
                "pluriel", "autres_sens", "score",
                "backend", "langue_detectee" } ]

Erreurs 400 k invalide, domaine inconnu, backend inconnu
        409 index sémantique demandé mais non construit
```

GET /api/translate

```
Paramètres  q  phrase à gloser (obligatoire, 500 caractères maxi)

Réponse     {
  "phrase", "glose", "couverture", "n_expressions",
  "non_resolus": [],
  "segments": [ { "source", "type", "n_mots", "confiance",
                  "lemme", "candidats": [] } ],
  "idiomes": [ { "mot_moore", "definition_francais",
                  "categorie", "score", "backend" } ],
```

```
"avertissement": "Glose mot-à-mot alignée sur le français..."
}
```

Note type vaut « expression », « mot », « ignore » ou « inconnu ».
« ignore » désigne un article ou une préposition sans
équivalent isolé en mooré.

GET /api/health

Réponse { "status": "ok",
 "entrees": 10566,
 "backends": ["tfidf", "entrees", "semantique", "hybride", "auto"],
 "backend_defaut": "auto",
 "semantique_disponible": true }

Références bibliographiques

Ressources linguistiques

1. SIL International. *Webonary Moore*, dictionnaire mooré, français et anglais en ligne. Source du corpus employé dans ce projet. <https://www.webonary.org/moore/>
2. EBERHARD, D. M., SIMONS, G. F., FENNIG, C. D. (dir.). *Ethnologue : Languages of the World*, notice « Mooré (mos) ». SIL International. <https://www.ethnologue.com/language/mos/>
3. Wikimedia Foundation. *Wikipidiya*, Wikipédia en langue mooré. <https://mos.wikipedia.org/>

Recherche d'information et représentation vectorielle

4. SALTON, G., BUCKLEY, C. *Term-weighting approaches in automatic text retrieval*. Information Processing and Management, vol. 24, n° 5, 1988, p. 513 à 523.
5. DEERWESTER, S., DUMAIS, S. T., FURNAS, G. W., LANDAUER, T. K., HARSHMAN, R. *Indexing by Latent Semantic Analysis*. Journal of the American Society for Information Science, vol. 41, n° 6, 1990, p. 391 à 407.
6. CORMACK, G. V., CLARKE, C. L. A., BUETTCHER, S. *Reciprocal Rank Fusion outperforms Condorcet and individual rank learning methods*. Actes de la conférence SIGIR, 2009, p. 758 à 759.
7. JOHNSON, J., DOUZE, M., JÉGOU, H. *Billion-scale similarity search with GPUs*. IEEE Transactions on Big Data, 2019. <https://arxiv.org/abs/1702.08734>

Modèles de langue et plongements lexicaux

8. REIMERS, N., GUREVYCH, I. *Sentence-BERT : Sentence Embeddings using Siamese BERT-Networks*. Actes de la conférence EMNLP-IJCNLP, 2019. <https://arxiv.org/abs/1908.10084>
9. REIMERS, N., GUREVYCH, I. *Making Monolingual Sentence Embeddings Multilingual using Knowledge Distillation*. Actes de la conférence EMNLP, 2020. Article décrivant la méthode dont est issu le modèle paraphrase-multilingual-MiniLM-L12-v2 employé ici. <https://arxiv.org/abs/2004.09813>

Outils et bibliothèques

10. PEDREGOSA, F. et coll. *Scikit-learn : Machine Learning in Python*. Journal of Machine Learning Research, vol. 12, 2011, p. 2825 à 2830. <https://jmlr.org/papers/v12/pedregosa11a.html>
11. Facebook AI Research. *FAISS, a library for efficient similarity search and clustering of dense vectors*. <https://github.com/facebookresearch/faiss>
12. Pallets. *Flask, documentation officielle*. <https://flask.palletsprojects.com/>
13. Wikimedia Foundation. *MediaWiki Action API : module de recherche*. Interface employée pour interroger les trois sources Web du projet. <https://www.mediawiki.org/wiki/API:Search>

Les ressources en ligne ont été consultées le 6 septembre 2026. Le site Webonary est protégé par un dispositif anti-robot : son accès est possible depuis un navigateur ordinaire mais refusé aux requêtes automatisées.